

you choose a language that is platform-independent to write extensions, in order to avoid recompilation of code for each platform. Extensions written in Basic are considered as macros, and were discussed in the article in LFY's May 2009 issue, *Automate Your Work with OpenOffice.org Macros*, Page 52. This month I will explain programming with Java, with the help of the NetBeans plug-in.

The OOo SDK

The SDK is the key prerequisite for writing extensions. It provides tools, the build environment, and the documentation required for programming. It also provides a good set of examples to start learning. It is available as a separate download for various platforms like Linux, Solaris, Mac, Windows, etc. The package repositories of some of the distros also include it. The SDK provides detailed documentation of APIs in the form of the Developer Guide. Some languages are well supported by the SDK and some are under development.

The SDK will be installed in any of the following possible paths:

- /usr/lib/openoffice/basis3.1/sdk
- /opt/openoffice.org/basis3.1/sdk

The SDK configuration is not covered in this article, as setting up the NetBeans plug-in will automatically take care of it.

Setting up the IDE

Download and install the latest version of the NetBeans IDE—the current version as of today is 6.5.

Go to **Tools→Plugins→Available plugins**. Select **OpenOffice.org API Plugin**, download and install it—currently, 2.0.4 is the latest version.

Go to **Tools→Options→Miscellaneous→OOo API Plugin**. By default, the SDK path is detected automatically—if so, verify it; else, provide the correct path.

Now the SDK and NetBeans are configured properly, and we're ready to get started with our extensions development.

Types of projects

You can create four types of projects with the help of this plug-in:

- **OpenOffice.org Add-On:** An add-on is a UI extension in the form of a menu item or tool bar item.
 - **OpenOffice.org Calc Add-In:** A Calc Add-In provides custom functions to spreadsheets.
 - **OpenOffice.org Component:** It helps to develop UNO-based applications.
 - **OpenOffice.org Client Application:** It helps to create client applications to bootstrap UNO and get a reference for running an office instance locally or remotely.
- In this article we'll discuss the first two types of projects.

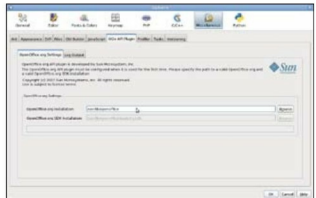


Figure 1: Setting up the OOo SDK

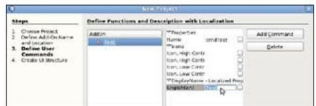


Figure 2: Add-on wizard: Command properties

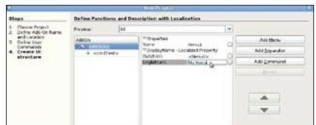


Figure 3: Add-on wizard: Menu properties

A simple add-on project

In this example, we'll create a simple add-on, which will load a new Calc document, insert a new sheet into it with the name "Hello", and change the contents of the A1 cell to 20.

Start the Add-on wizard using **File → New Project → OpenOffice.org → OpenOffice.org Add-on**. In the next screen, fill in the project name, say "Simple", which will be the default name for the *Main* class also. Then fill in a suitable package name, say *org.lfy.example*, and a suitable location to store the project. Finally, select whether the add-on should come as a menu item, a toolbar item or both.

In the next screen (Figure 2) give a suitable name for the command, say *cmdTest* in place of 'Command0', which is there by default, and the display name as *Test*. We can leave the 'Icon' field empty for the moment. Note that we can create more than one command using the **Add Command** button; however, in this example we will restrict ourselves to one command only.

In the next screen, we can create the menu properties, such as its name, etc (Figure 3).

On the same screen and the next one, select the